UNIVERSIDADE NOVE DE JULHO – UNINOVE
DIRETORIA DE INFORMÁTICA
CURSO CIÊNCIA DA COMPUTAÇÃO

NICHOLAS MONTEIRO DOS SANTOS – RA:2224200661

AGENDA UBS: SISTEMA WEB DE AGENDAMENTO DE CONSULTAS PARA POSTO DE SAÚDE COMUNITÁRIO

Organização da agenda e acompanhamento do absenteísmo na Atenção Primária

SÃO PAULO
2026

NICHOLAS MONTEIRO DOS SANTOS – RA: 2224200662

AGENDA UBS: SISTEMA WEB DE AGENDAMENTO DE CONSULTAS PARA POSTO DE SAÚDE COMUNITÁRIO

Organização da agenda e acompanhamento do absenteísmo na Atenção Primária

Projeto de Extensão apresentado ao Curso de Ciência da Computação da Universidade Nove de Julho (UNINOVE), como requisito parcial para aprovação na disciplina de Projeto de Extensão em Desenvolvimento de Software.

Professora Orientadora: Prof.ª Dra./Me. Debora V. Canne.

SÃO PAULO
2026

RESUMO

Este trabalho descreve o Agenda UBS, um sistema web de agendamento de consultas feito como Projeto de Extensão para um posto de saúde comunitário. A ideia partiu de um problema conhecido da Atenção Primária: as faltas dos pacientes, que deixam horários vazios e aumentam a fila. O sistema permite à recepção cadastrar pacientes e profissionais, liberar horários, marcar e cancelar consultas e registrar quem compareceu, e com esses registros calcula a taxa de faltas em um painel. Foi desenvolvido em Next.js, TypeScript e Supabase (PostgreSQL), em fases curtas com testes manuais a cada entrega. As regras mais importantes do agendamento ficaram no próprio banco de dados, junto com o controle de acesso por perfil e os cuidados da LGPD. O resultado é um sistema funcional, publicado na internet, com interface de letras grandes e alto contraste. Ele ainda não foi testado pela equipe de um posto real, que é o passo seguinte, ao lado dos lembretes e da confirmação feita pelo paciente.

Palavras-chave: Desenvolvimento de Software. Projeto de Extensão. Atenção Primária. Agendamento de Consultas. Acessibilidade. UNINOVE.

SUMÁRIO

1. INTRODUÇÃO **5**

1.1 CONTEXTUALIZAÇÃO E JUSTIFICATIVA DE EXTENSÃO 5

1.2 PROBLEMATIZAÇÃO 6

1.3 OBJETIVOS 6

1.3.1 Objetivo Geral 6

1.3.2 Objetivos Específicos 6

2. REVISÃO BIBLIOGRÁFICA E FUNDAMENTAÇÃO TEÓRICA **7**

3. COMUNIDADE ATENDIDA E IMPACTO SOCIAL **9**

4. ESPECIFICAÇÃO DO SISTEMA E REQUISITOS **10**

4.1 REQUISITOS FUNCIONAIS (RF) 10

4.2 REQUISITOS NÃO FUNCIONAIS (RNF) 11

4.3 ARQUITETURA DO SISTEMA E TECNOLOGIAS UTILIZADAS 11

4.4 MODELAGEM E DIAGRAMAS DE SISTEMA 13

5. METODOLOGIA DE DESENVOLVIMENTO E CRONOGRAMA **15**

6. RESULTADOS OBTIDOS E TESTES DE SOFTWARE **16**

6.1 TELAS DESENVOLVIDAS **16**

6.2 TESTES REALIZADOS **21**

6.3 REVISÃO DE SEGURANÇA **22**

7. CONSIDERAÇÕES FINAIS **23**

REFERÊNCIAS **24**

1. INTRODUÇÃO

Quem já precisou de uma consulta no posto de saúde sabe que o horário é disputado. Por isso, quando o paciente marca e não aparece, a perda é de todos: aquela vaga poderia ter atendido outra pessoa. Os estudos que encontrei mostram que a taxa de faltas muda bastante de uma unidade para outra, o que sugere que a forma como o posto organiza a agenda influencia o resultado (MORAIS et al., 2023).

Este trabalho apresenta o **Agenda UBS**, um sistema web de agendamento para uma Unidade Básica de Saúde (UBS) ou posto de saúde comunitário. Quem usa o sistema é a recepção: ela cadastra pacientes e profissionais, libera horários, marca e cancela consultas e anota se o paciente veio ou faltou. Com essas anotações, o próprio sistema calcula a taxa de faltas e mostra o resultado em um painel, para que a equipe enxergue o problema em números.

Fiz o projeto sozinho, na disciplina de Projeto de Extensão em Desenvolvimento de Software do curso de Ciência da Computação da Universidade Nove de Julho (UNINOVE). O código está em um repositório público (https://github.com/Nicholas866/agendamento-posto-de-saude-comunitario) e há uma versão no ar em https://agendamento-posto-de-saude-comunita.vercel.app. Todos os dados de exemplo (pacientes, documentos, telefones e e-mails) são inventados.

1.1 CONTEXTUALIZAÇÃO E JUSTIFICATIVA DE EXTENSÃO

A extensão universitária serve para levar o que se aprende em sala de aula para fora dela, em contato com a comunidade. As Diretrizes para a Extensão na Educação Superior determinam que pelo menos 10% da carga horária dos cursos de graduação seja dedicada a atividades desse tipo (BRASIL, 2018a). Nesta disciplina, a extensão acontece quando o aluno escolhe um problema real e tenta resolvê-lo com software.

Escolhi o posto de saúde porque a agenda de consultas é um problema concreto e que reúne muita coisa que estudei no curso: levantamento de requisitos, banco de dados, desenvolvimento web, segurança e proteção de dados. Procurei fazer uma ferramenta simples, pensada para a rotina da recepção, com letras grandes, bastante contraste e poucos cliques, porque nem todo mundo que vai usá-la tem facilidade com tecnologia.

1.2 PROBLEMATIZAÇÃO

Quando uma consulta é marcada e o paciente não aparece, o posto perde a vaga, a fila anda mais devagar e ninguém sabe ao certo o tamanho do problema. Em muitos lugares a agenda ainda é feita em papel ou em planilha, e fica difícil saber quais horários estão livres, evitar duas marcações no mesmo horário e contar quantas faltas aconteceram.

Já existem iniciativas oficiais de agendamento no SUS, mas a pergunta deste projeto é mais específica: **como dar à recepção de um posto comunitário uma ferramenta simples para organizar a agenda local, evitar conflitos de horário e registrar as faltas, de modo que a própria equipe consiga calcular e acompanhar sua taxa de absenteísmo?**

1.3 OBJETIVOS

1.3.1 Objetivo Geral

Desenvolver um sistema web de agendamento de consultas para um posto de saúde comunitário, que seja simples de usar, evite conflitos de horário e permita acompanhar a taxa de faltas dos pacientes.

1.3.2 Objetivos Específicos

- Levantar os requisitos funcionais e não funcionais do sistema.
- Modelar os dados e colocar no banco as regras mais importantes do agendamento.
- Implementar o sistema com uma arquitetura web atual e acesso por perfil: recepção, profissional de saúde e administrador.
- Cuidar da usabilidade e da acessibilidade, com letras grandes, bom contraste e status mostrado em cor e em texto.
- Tratar os dados de acordo com a LGPD, coletando o mínimo e controlando quem acessa o quê.
- Testar cada etapa e registrar o que funcionou, o que deu errado e o que ficou para depois.

2. REVISÃO BIBLIOGRÁFICA E FUNDAMENTAÇÃO TEÓRICA

Absenteísmo na Atenção Primária

Absenteísmo é o nome técnico para o paciente que marca a consulta e não comparece. Morais et al. (2023) estudaram o tema na Atenção Primária de Ribeirão Preto (SP) e contam que o município tem taxas perto de 30%. Segundo os autores, a literatura cita para o SUS valores próximos ou acima de 25%, contra uma média mundial de 23%. Eles tratam o assunto principalmente como uma questão de gestão do serviço, e não só de comportamento do paciente.

Os números mudam conforme o lugar e o período. A Prefeitura de São José dos Campos (2023) informou que, no primeiro trimestre de 2022, foram agendadas 171.135 consultas nas UBSs e 26.985 tiveram falta (15,77%). No primeiro trimestre de 2023, foram 199.503 consultas e 27.777 faltas (13,37%). Esses dados mostram que a taxa de faltas pode ser medida e acompanhada, e medir é o primeiro passo para qualquer tentativa de reduzi-la. Por isso o Agenda UBS não promete reduzir as faltas em um percentual qualquer: ele registra e mede.

Engenharia de software e requisitos

Em engenharia de software, o desenvolvimento passa por comunicação, planejamento, modelagem, construção e entrega (PRESSMAN; MAXIM, 2016). Os requisitos dizem o que o sistema deve fazer (funcionais) e como ele deve se comportar, por exemplo quanto à segurança e à facilidade de uso (não funcionais) (SOMMERVILLE, 2018). Os diagramas que aparecem neste trabalho, de casos de uso, de dados e de estados, serviram para organizar as regras antes de programá-las e para conferir o resultado depois.

Tecnologias utilizadas

O sistema foi feito com **Next.js** e **React**, que permitem montar as páginas no servidor e executar as operações de gravação em funções que rodam só no servidor, as *Server Actions* (NEXT.JS, 2026). A linguagem é o **TypeScript**, que acrescenta tipos ao JavaScript e ajuda a pegar erros antes de rodar o código. Para guardar os dados e fazer o login usei o **Supabase**, que oferece um banco **PostgreSQL** e um serviço de autenticação (SUPABASE, 2026). Um recurso importante do PostgreSQL neste projeto é a *Row Level Security* (RLS), que define no próprio banco quais linhas cada usuário pode ler ou alterar (POSTGRESQL GLOBAL

DEVELOPMENT GROUP, 2026). Com isso, se alguma checagem da tela falhar, o banco ainda barra o acesso indevido.

Proteção de dados e acessibilidade

A LGPD considera dado pessoal sensível aquele referente à saúde (BRASIL, 2018b, art. 5º, inciso II). A lei permite tratar esse tipo de dado, entre outras situações, para a tutela da saúde, em procedimentos feitos por profissionais ou serviços de saúde (art. 11, inciso II, alínea f). Também valem os princípios da finalidade e da necessidade, que pedem para coletar só o que é indispensável, e por isso o sistema não guarda diagnóstico nem prontuário. Na acessibilidade, usei como referência as Diretrizes de Acessibilidade para Conteúdo Web (W3C, 2018): bom contraste de cores, texto legível e não depender só da cor para passar uma informação.

3. COMUNIDADE ATENDIDA E IMPACTO SOCIAL

O público beneficiado é a comunidade atendida por um posto de saúde comunitário, principalmente quem depende do SUS para consultas de rotina: hipertensos, diabéticos, gestantes, crianças e idosos. Quem usa o sistema no dia a dia é a equipe do posto: a **recepção**, que cuida da agenda; os **profissionais de saúde**, que consultam a agenda e os atendimentos; e o **administrador**, que cadastra profissionais e usuários. O posto onde o sistema seria implantado ainda está **a definir**.

Uma decisão importante foi que o paciente **não precisa de celular, de aplicativo nem de login**. Quem mexe no sistema é a recepção, que marca e cancela as consultas em nome dele, inclusive quando ele liga ou aparece no balcão. A tela usa letras grandes, bastante contraste e mensagens em português simples, pensando em quem trabalha na recepção e também em possíveis usuários idosos.

O impacto social esperado é indireto e vem de três coisas: a agenda fica mais organizada, com um paciente por horário; um horário cancelado volta a ficar livre e pode ser usado por outra pessoa; e a equipe passa a ter números sobre as faltas para discutir o que fazer. Como o sistema ainda não foi usado em um posto de verdade, **não há medição de impacto**. Comparar a taxa de faltas antes e depois do uso fica como trabalho futuro (seção 7).

4. ESPECIFICAÇÃO DO SISTEMA E REQUISITOS

4.1 REQUISITOS FUNCIONAIS (RF)

- RF-001: O sistema deve autenticar usuários por e-mail e senha e conceder acesso conforme o perfil (recepção, profissional ou administrador); quem não tem perfil cadastrado não acessa o sistema.
- RF-002: O sistema deve permitir cadastrar, editar e buscar pacientes por nome, CPF ou Cartão SUS.
- RF-003: O sistema deve permitir cadastrar, editar e ativar/desativar profissionais, sem apagá-los, para preservar o histórico.
- RF-004: O sistema deve permitir liberar vários horários de uma vez para um profissional ativo, informando data, intervalo e duração (de 15 a 60 minutos), sem duplicar ou sobrepor horários.
- RF-005: O sistema deve permitir remover horários livres, preservando o histórico das consultas.
- RF-006: O sistema deve permitir marcar uma consulta escolhendo paciente, profissional ativo, dia e horário livre.
- RF-007: O sistema deve exibir a agenda do dia, agrupada por profissional, com navegação entre dias e status em cor e texto.
- RF-008: O sistema deve exibir o detalhe da consulta, com dados de contato visíveis apenas à recepção e ao administrador.
- RF-009: O sistema deve permitir cancelar consultas futuras, com confirmação, liberando o horário para nova marcação.
- RF-010: O sistema deve permitir marcar a consulta como Realizada ou Faltou até o dia da consulta, e desfazer essa marcação.
- RF-011: O sistema deve exibir um painel com totais por status, taxa de faltas, tabelas e gráficos por semana e por profissional, com filtro de período.
- RF-012: O sistema deve permitir que o administrador crie contas de usuário e defina o perfil de cada uma.

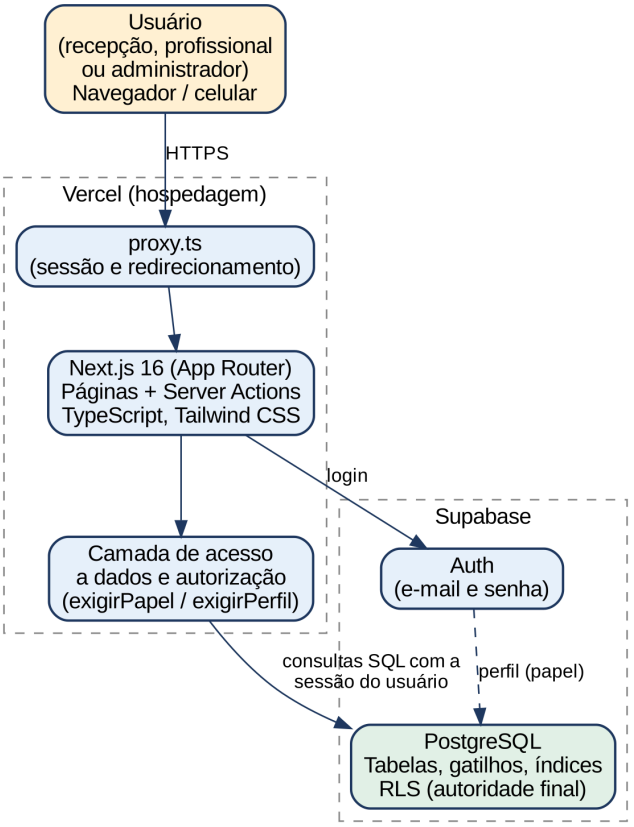
4.2 REQUISITOS NÃO FUNCIONAIS (RNF)

- RNF-001 (Usabilidade e acessibilidade): a interface deve ter letras grandes, bom contraste, mensagens de erro em português e indicar o status das consultas por cor e por texto.
- RNF-002 (Responsividade): a interface deve se adaptar a telas de computador e de celular.
- RNF-003 (Segurança): a autorização deve ser feita em camadas (redirecionamento, páginas, ações do servidor e RLS no banco); as chaves secretas devem ficar apenas no servidor e fora do controle de versão; não há cadastro público de usuários.
- RNF-004 (Integridade): as regras críticas de agendamento devem ser impostas no banco de dados, e não apenas na tela.
- RNF-005 (Privacidade – LGPD): coletar o mínimo de dados, não registrar diagnóstico nem prontuário, não registrar dados pessoais em logs e usar somente dados fictícios em desenvolvimento e testes.
- RNF-006 (Fuso horário): datas e horas devem ser exibidas no fuso America/Sao_Paulo.
- RNF-007 (Desempenho): consultas de listas e do painel devem ser paginadas e agregadas no servidor, sem trazer dados pessoais de pacientes. O tempo de resposta não foi medido formalmente nesta versão.
- RNF-008 (Manutenibilidade): código em TypeScript, com verificação de estilo (lint) e compilação de produção a cada etapa.

4.3 ARQUITETURA DO SISTEMA E TECNOLOGIAS UTILIZADAS

A Figura 1 mostra como o sistema está organizado. O usuário acessa o site hospedado na Vercel. Um arquivo de *proxy* renova a sessão e manda para o login quem não estiver autenticado. As páginas e as *Server Actions* do Next.js conferem o perfil do usuário antes de qualquer operação, e tudo o que é lido ou gravado passa pelo PostgreSQL do Supabase, onde as políticas de RLS têm a palavra final. As tecnologias estão listadas na Tabela 1.

Figura 1 – Arquitetura do sistema Agenda UBS



Fonte: Elaborado pelo autor (2026).

Tabela 1 – Tecnologias utilizadas

Finalidade	**Ferramenta**
Aplicação web	Next.js 16 (App Router), React e TypeScript
Estilo da interface	Tailwind CSS v4
Banco de dados e autenticação	Supabase (PostgreSQL, Auth e Row Level Security)
Hospedagem	Vercel (plano gratuito)
Controle de versão	Git e GitHub
Apoio de IA no desenvolvimento	Cursor e OpenCode, com revisão e teste manual de cada etapa

Fonte: Elaborado pelo autor (2026).

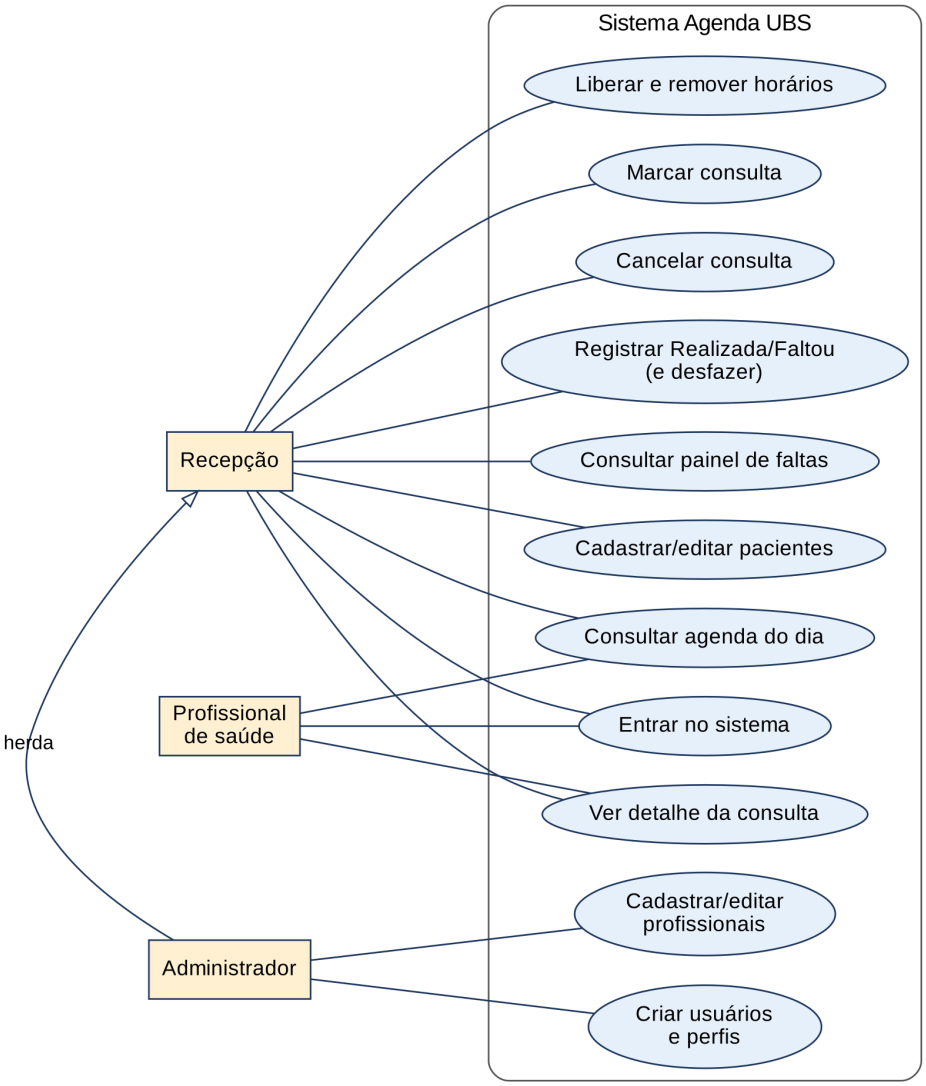
As regras de negócio mais importantes ficaram no banco de dados, em índices, gatilhos e funções, e não só na tela: (i) um índice único parcial impede dois pacientes no mesmo horário, ignorando as consultas canceladas; (ii) um gatilho barra agendamentos no passado ou com mais de 30 dias de antecedência; (iii) outro gatilho recalcula o status do horário a partir das consultas ativas, e por isso cancelar libera o horário; (iv) horários removidos não são apagados de verdade (exclusão lógica). Duas regras ficaram só na

aplicação: marcar Realizada ou Faltou apenas até o dia da consulta e não liberar horários para profissionais desativados.

4.4 MODELAGEM E DIAGRAMAS DE SISTEMA

A Figura 2 é o diagrama de casos de uso, com os três perfis de usuário. O administrador faz tudo o que a recepção faz e ainda cadastra profissionais e cria usuários. O profissional de saúde só consulta. O paciente não aparece como ator porque não tem login.

Figura 2 – Diagrama de casos de uso

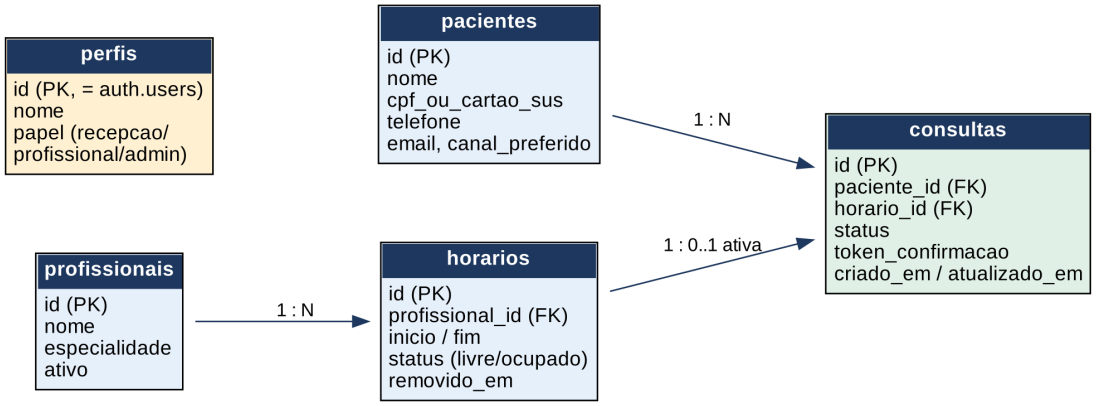


Fonte: Elaborado pelo autor (2026).

A Figura 3 mostra o modelo de dados. Um profissional tem vários horários, cada horário pode ter no máximo uma consulta ativa e cada paciente pode ter várias consultas. A tabela de perfis se liga ao usuário do serviço de login e guarda o papel de cada pessoa. A

consulta também tem um token de confirmação, que deixei preparado para uma futura confirmação pelo paciente por link, mas essa página pública ainda não existe (seção 7).

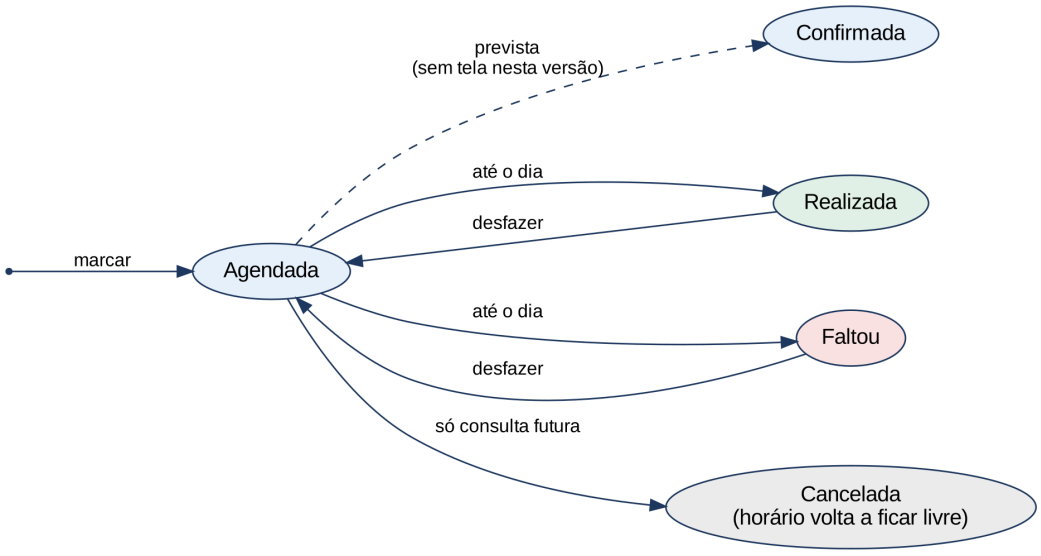
Figura 3 – Modelo de dados (entidades e relacionamentos)



Fonte: Elaborado pelo autor (2026).

A Figura 4 mostra o ciclo de vida de uma consulta. Ela começa como Agendada e pode ser cancelada, só se for futura, ou marcada como Realizada ou Faltou, só até o dia da consulta. A recepção e o administrador podem desfazer o Realizada ou Faltou, e a consulta volta a Agendada. O status Confirmada existe no banco, mas depende da confirmação pelo paciente, que não foi implementada.

Figura 4 – Diagrama de estados da consulta



Fonte: Elaborado pelo autor (2026).

As telas do sistema aparecem na seção 6. Não usei o Figma: as telas foram definidas direto no código, a partir do plano de fases, e ajustadas conforme os testes.

5. METODOLOGIA DE DESENVOLVIMENTO E CRONOGRAMA

Trabalhei sozinho e, por isso, fui ao mesmo tempo analista, desenvolvedor e testador, com a orientação da professora da disciplina. Não usei um framework formal como o Scrum. Fui por fases curtas, definidas em um plano (o arquivo PLANO.md do repositório), seguindo uma abordagem **iterativa e incremental** e usando uma lista simples de tarefas (a fazer, em andamento, concluído), parecida com um Kanban. No fim de cada fase eu rodava a verificação de código (*lint*), gerava a versão de produção e fazia um roteiro de testes manuais. Só depois de tudo aprovado o código ia para o GitHub, com commits curtos em português.

Escrevi o código com ajuda de ferramentas de inteligência artificial (Cursor e OpenCode). Eu definia os requisitos e as regras de negócio, passava as instruções de cada fase, lia o que a ferramenta tinha feito e fazia os testes. Em vários momentos foi preciso corrigir o resultado, como mostram os erros achados nos testes e na revisão de segurança feita por um colega, descritos na seção 6.

Tabela 2 – Fases de desenvolvimento

Fase	Entrega	Situação
0	Estrutura do projeto: Next.js, Tailwind e conexão com o Supabase	Concluída e testada
1	Banco de dados (tabelas, regras, RLS) e login por perfil	Concluída e testada
2	Cadastros de pacientes e profissionais	Concluída e testada
3A	Horários, nova consulta e agenda do dia	Concluída e testada
3B	Detalhe da consulta, cancelamento e marcação de presença/falta com desfazer	Concluída e testada
5	Painel de faltas (totais, taxa, gráficos e tabelas)	Concluída e testada
Extra	Tela de usuários para o administrador; revisão de segurança; publicação na Vercel	Concluída
Futuro	Remarcar consulta, lembretes, confirmação pelo paciente e demais itens da seção 7	Não realizada

Tabela 3 – Cronograma acadêmico da disciplina

Período	Atividade
10/08/2026	Início do projeto; formação do grupo e pesquisa de temas
Até 01/09/2026	Definição do tema; estudo de ferramentas (Git e GitHub)
Setembro de 2026	Planejamento e distribuição de tarefas
Outubro e novembro de 2026	Pesquisa e desenvolvimento do projeto
Até 27/11/2026	Entrega final (limite máximo)

6. RESULTADOS OBTIDOS E TESTES DE SOFTWARE

O sistema está pronto nas funcionalidades principais e publicado na Vercel, em https://agendamento-posto-de-saude-comunita.vercel.app. Os dados de acesso dos usuários de demonstração (administrador, recepção e profissional) são enviados junto com a entrega do trabalho e não aparecem aqui nem no repositório. O código está em https://github.com/Nicholas866/agendamento-posto-de-saude-comunitario.

6.1 TELAS DESENVOLVIDAS

As Figuras 5 a 11 mostram as principais telas do sistema funcionando.

Figura 5 – Tela de login



Figura 6 – Agenda do dia, com status em cor e texto

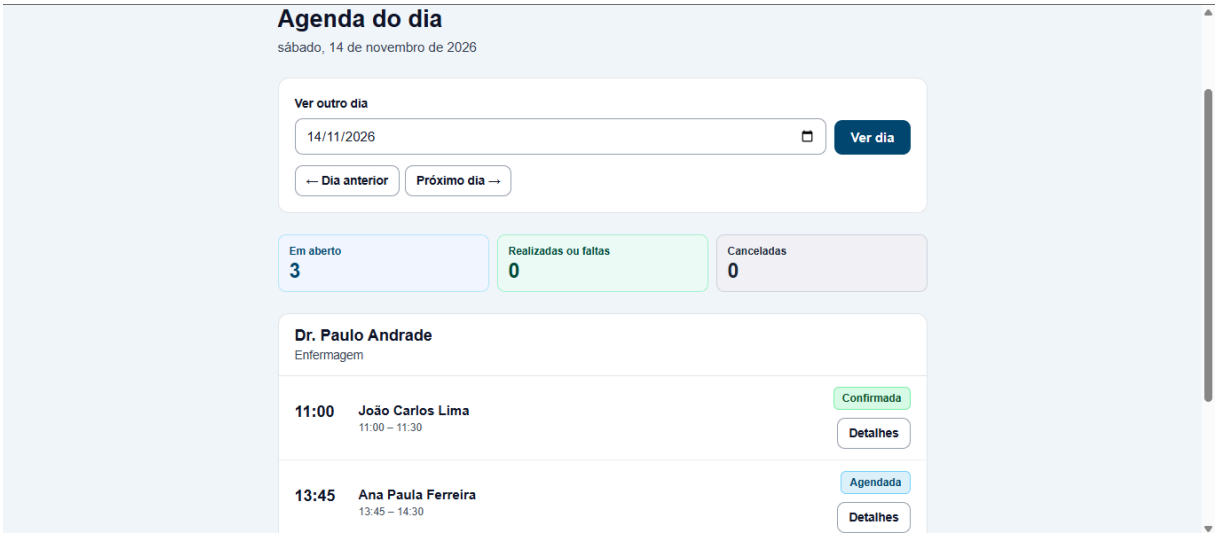


Figura 7 – Liberação de horários e nova consulta

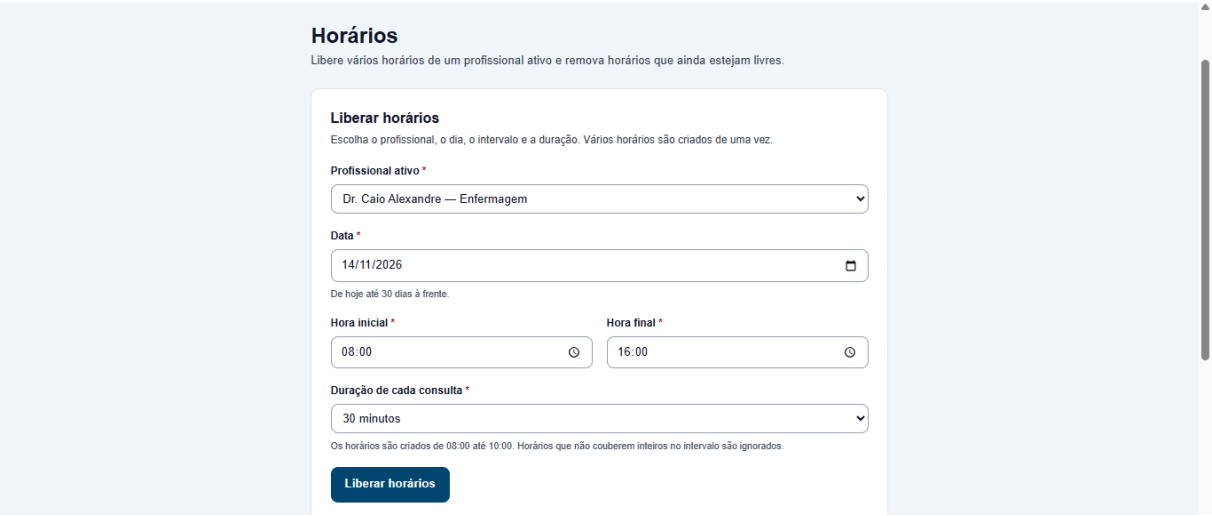


Figura 8 – Detalhe da consulta, com ações de cancelar e registrar presença

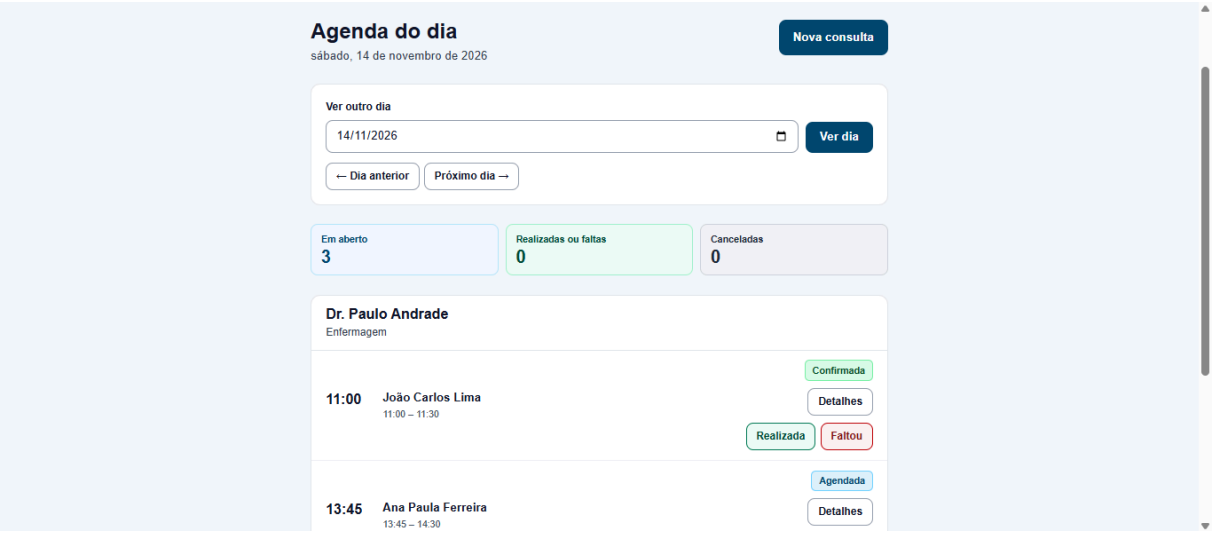


Figura 9 – Painel de faltas, com taxa, gráficos e tabelas

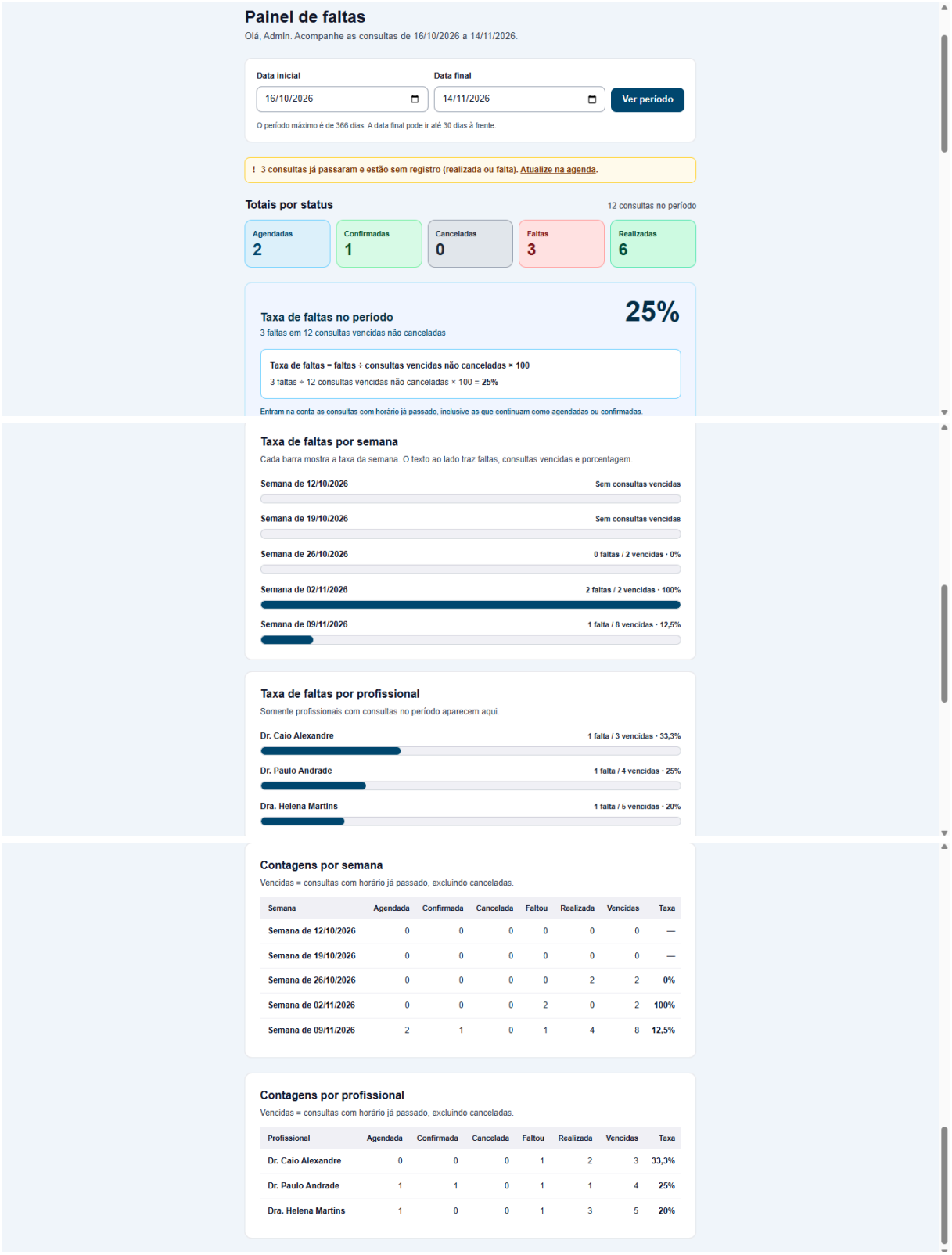


Figura 10 – Registro e cadastro dos pacientes/profissionais

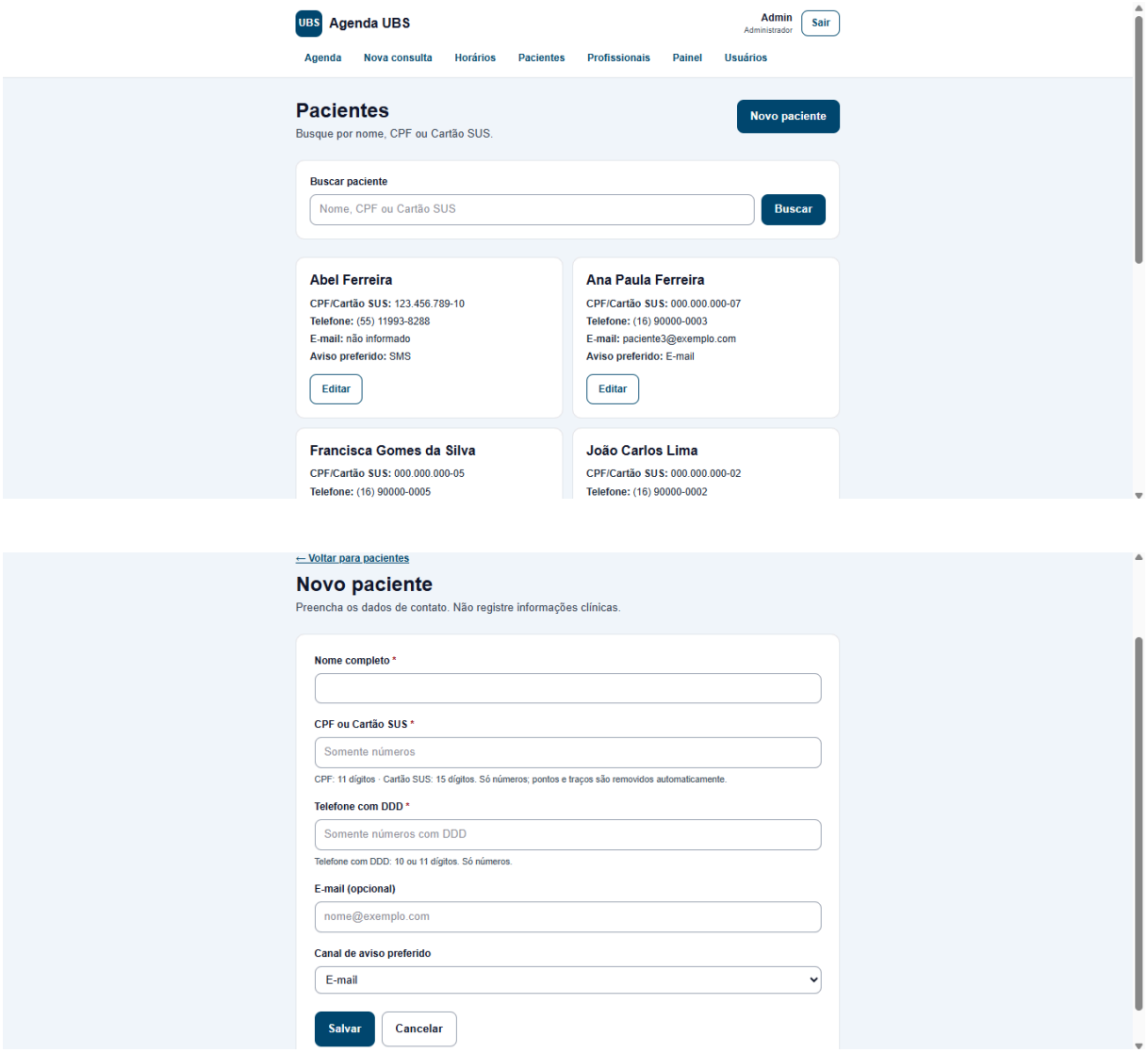


Figura 11 – Registro e cadastro dos profissionais

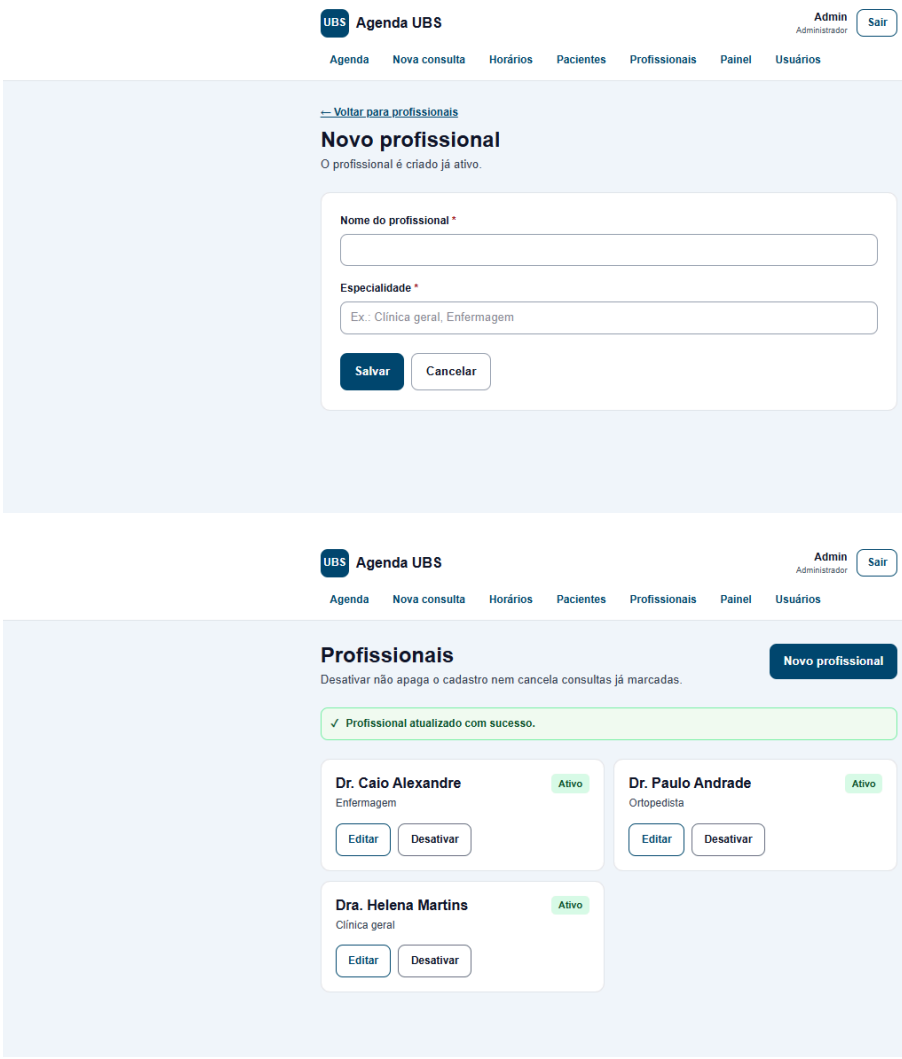
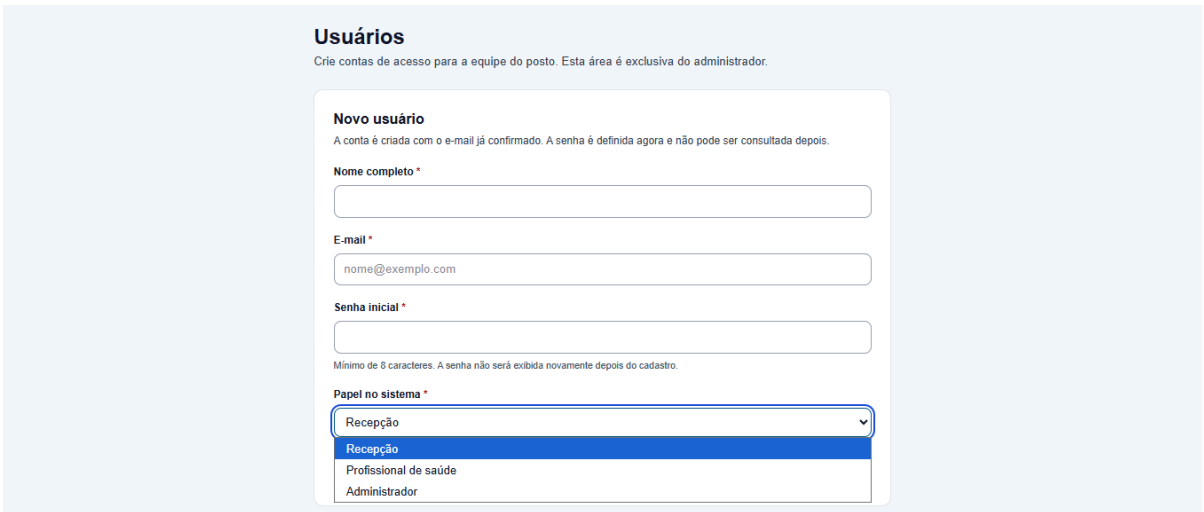


Figura 12 – Cadastro de usuários da UBS



6.2 TESTES REALIZADOS

Não montei testes automatizados. A validação combinou a verificação de código (*lint*), a geração da versão de produção e roteiros de teste manuais, feitos por mim a cada fase, sempre com dados fictícios. A Tabela 4 resume os principais casos.

Tabela 4 – Principais casos de teste manual

Caso de teste	Resultado esperado	Resultado
Entrar com usuário sem perfil cadastrado	Acesso negado, com mensagem em português	Aprovado
Cadastrar paciente com telefone ou documento fora do limite de dígitos	Erro de validação; cadastro não é salvo	Aprovado
Liberar horários sobrepostos ou duplicados	Sistema não cria horários repetidos	Aprovado
Marcar consulta em data anterior a hoje	Marcação bloqueada	Aprovado
Marcar dois pacientes no mesmo horário	Segundo agendamento recusado pelo banco	Aprovado
Cancelar consulta futura e abrir Nova consulta	Horário volta a ficar livre	Aprovado após correção*
Marcar Realizada/Faltou em consulta de data futura	Mensagem: "Só é possível marcar presença ou falta até o dia da consulta."	Aprovado
Desfazer Realizada/Faltou em consulta de hoje ou passada	Status volta a Agendada	Aprovado
Entrar como profissional	Vê agenda e detalhe sem contato do paciente e sem botões de ação	Aprovado
Alterar o período do painel e marcar uma falta	Totais e taxa são recalculados	Aprovado
Período sem dados no painel	Estado vazio com orientação	Aprovado

No teste de cancelamento, o horário cancelado **não** voltou a aparecer como livre. O problema estava no gatilho que atualiza o status do horário. Ele foi reescrito para recalcular o status a partir das consultas ativas (arquivo supabase/fase3b.sql) e o teste foi repetido, dessa vez com sucesso. Foi um bom exemplo de por que vale testar cada fase e deixar as regras no banco.

6.3 REVISÃO DE SEGURANÇA

Depois da publicação, um colega fez uma revisão geral do projeto, e dela saíram várias correções: as políticas de leitura (RLS) passaram a exigir um perfil cadastrado; o cadastro público de usuários foi desativado no Supabase; o aviso de conexão com o banco saiu da página inicial; e foi criada a tela de usuários para o administrador, que usa uma chave de serviço lida só no servidor, em um módulo restrito (pacote *server-only*), fora do Git e guardada como variável secreta na Vercel. A documentação também foi corrigida: ela dizia que a restrição dos dados de contato para o profissional era feita no banco, mas na versão atual essa restrição existe só na tela, e fazê-la por coluna no banco ficou como trabalho futuro.

7. CONSIDERAÇÕES FINAIS

A Agenda UBS cumpriu o objetivo de entregar um sistema de agendamento funcional, de uso simples e publicado na internet. Ele cobre o fluxo principal da recepção de um posto: cadastrar pacientes e profissionais, liberar horários, marcar e cancelar consultas, registrar presença ou falta e acompanhar a taxa de faltas em um painel. As regras críticas estão no banco de dados e a autorização é feita em camadas, o que dá mais segurança do que depender só da tela.

Aprendi bastante ao longo do caminho. Na parte técnica, passei por levantamento de requisitos, modelagem de dados, desenvolvimento web com TypeScript, segurança com RLS, LGPD, publicação na nuvem e controle de versão com Git. Na parte de extensão, o projeto me fez pensar em quem vai usar o sistema: um sistema feito para a recepção, e não para o paciente, tem mais chance de ser usado de verdade. Sobre as ferramentas de IA, elas aceleraram a escrita do código, mas nada pode ser aceito sem revisão, e os erros que apareceram nos testes e na revisão de segurança mostram isso.

O que ficou de fora desta versão e pode ser feito no futuro:

- remarcar consulta (por enquanto é preciso cancelar e marcar de novo);
- lembrete automático por e-mail, WhatsApp ou SMS e confirmação pelo paciente por link (o banco já tem as funções de apoio, mas faltam a página pública e o envio);
- lista de espera para horários liberados e histórico de alterações de cada consulta;
- restrição dos dados de contato no próprio banco para o perfil de profissional e vínculo entre o usuário profissional e o seu cadastro, para mostrar só a agenda dele;
- implantação em um posto real, com a equipe testando o sistema e a taxa de faltas medida antes e depois;
- numa implementação real, ainda seria preciso definir quem atende os pedidos dos titulares dos dados, ter um plano para incidentes e, se for essa a base legal, um termo de consentimento.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: Informação e documentação – Referências – Elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: Informação e documentação – Citações em documentos – Apresentação. Rio de Janeiro: ABNT, 2002.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação – Trabalhos acadêmicos – Apresentação. Rio de Janeiro: ABNT, 2011.

BRASIL. Ministério da Educação. Conselho Nacional de Educação. Câmara de Educação Superior. **Resolução nº 7, de 18 de dezembro de 2018**. Estabelece as Diretrizes para a Extensão na Educação Superior Brasileira. Diário Oficial da União, Brasília, DF, 19 dez. 2018a.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União, Brasília, DF, 15 ago. 2018b.

MORAIS, Rinaldo Macedo de; SISDELLI, Marina Fröner; FERREIRA, Gabriela Souza Assis; COSTA, André Lucirton. Gestão do absenteísmo na Atenção Primária em cidade brasileira de médio porte. **Interface (Botucatu)**, v. 27, e220197, 2023. Disponível em: https://scielosp.org/pdf/icse/2023.v27/e220197/pt. Acesso em: 14 nov. 2026.

NEXT.JS. **Next.js Documentation**. Vercel, 2026. Disponível em: https://nextjs.org/docs. Acesso em: 14 nov. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation**: Row Security Policies. 2026. Disponível em: https://www.postgresql.org/docs/current/ddl-rowsecurity.html. Acesso em: 14 nov. 2026.

PREFEITURA DE SÃO JOSÉ DOS CAMPOS. **São José reduz taxa de faltas em consultas nas UBSs Resolve**. São José dos Campos, 7 jun. 2023. Disponível em: https://www.sjc.sp.gov.br/noticias/2023/junho/07/sao-jose-reduz-taxa-de-faltas-em-consultas-nas-ubss-resolve/. Acesso em: 14 nov. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SANTOS, Nicholas Monteiro dos. **Agenda UBS**: sistema de agendamento para posto de saúde comunitário. GitHub, 2026. Disponível em: https://github.com/Nicholas866/agendamento-posto-de-saude-comunitario. Acesso em: 14 nov. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.

SUPABASE. **Supabase Documentation**. 2026. Disponível em: https://supabase.com/docs. Acesso em: 14 nov. 2026.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.1**. 2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: 14 nov. 2026.